

VayuSphere: A Hyperspherical Centroid Adapter for Query-Key Modulation in Low-Resource Decoder Transformers

Divyang Panchasara

ASTRA-LM Research Project

Draft v0.1

Abstract

We introduce **VayuSphere**, a lightweight hyperspherical adapter for decoder-only language models. VayuSphere projects attention query and key vectors onto a unit hypersphere, compares them against a small bank of learnable angular centroids, and uses the resulting centroid similarity to modulate query/key magnitudes before scaled dot-product attention. The goal is to test whether angular structure in the query-key space can improve low-resource language modeling with minimal parameter overhead.

We evaluate VayuSphere in **ASTRA-LM**, a compact GPT-style decoder transformer using RoPE, RMSNorm, SwiGLU, and PyTorch scaled dot-product attention. Experiments are performed on a nano-scale model with approximately **17M parameters**, trained on a 10M-token FineWeb-Edu GPT-2-tokenized subset and evaluated with 200 validation batches, approximately **102,400 tokens** per evaluation.

Our strongest single-seed result shows that `post_rope + topk8 + trained centroids` improves evaluation loss over baseline by approximately **0.0069 nats**. However, a 3-seed proof test does not validate a robust baseline-beating claim: the best VayuSphere candidate obtains mean loss **6.2485**, compared with baseline mean loss **6.2470**. Despite this, VayuSphere consistently shows internal mechanism signal: trained top-k centroids outperform learned attention temperature, frozen random centroids, and all-centroid VayuSphere controls on average. These findings suggest that hyperspherical centroid routing contains useful signal, but the current scale-only implementation is not yet strong enough to robustly outperform a clean GPT baseline.

1. Introduction

Attention mechanisms in decoder-only transformers rely heavily on the geometry of query and key vectors. In standard scaled dot-product attention, token interactions are determined by the inner product between query and key representations. This creates a natural connection between attention behavior and angular geometry: after normalization, the dot product becomes cosine similarity on a hypersphere.

VayuSphere explores whether explicit hyperspherical structure can improve compact language models. Instead of replacing attention, VayuSphere acts as a lightweight adapter around query and key tensors. It introduces a small set of learnable centroids on the hypersphere and computes token-to-centroid similarity.

These similarities generate a content-dependent gate that modulates query and key vectors before attention.

The motivation is threefold:

1. **Low parameter overhead:** A centroid bank adds very few parameters compared with the embedding and transformer body.
2. **Attention geometry control:** Query-key interactions may benefit from angular grouping or top-k centroid specialization.
3. **Low-VRAM experimentation:** The method is compatible with ordinary PyTorch training on consumer GPUs, including 6GB laptop GPUs.

This paper reports the current state of VayuSphere v0.2. The results are intentionally presented conservatively. VayuSphere shows measurable mechanism signal, but does not yet establish a robust improvement over the GPT baseline.

2. Background

2.1 Decoder Transformer Baseline

ASTRA-LM uses a compact GPT-style decoder architecture with:

- token embeddings,
- RMSNorm pre-normalization,
- rotary positional embeddings,
- scaled dot-product attention,
- grouped-query-compatible attention paths,
- SwiGLU feed-forward layers,
- tied language modeling head.

The primary baseline is a nano-scale model with approximately **16.999M parameters**.

2.2 Query-Key Geometry

For query vector q and key vector k , standard attention logits are:

$$\ell(q, k) = \frac{q^\top k}{\sqrt{d}}$$

If vectors are normalized:

$$\hat{q} = \frac{q}{\|q\|}, \quad \hat{k} = \frac{k}{\|k\|}$$

then:

$$\hat{q}^\top \hat{k} = \cos(\theta)$$

where θ is the angular distance between query and key directions. VayuSphere uses this observation to introduce explicit angular centroids into the attention path.

3. VayuSphere Method

3.1 Hyperspherical Centroid Bank

VayuSphere maintains a centroid matrix:

$$C = \{c_1, c_2, \dots, c_m\}$$

where each centroid $c_i \in \mathbb{R}^{d_h}$, and d_h is the attention head dimension. In the current nano configuration, VayuSphere adds only about **6,144 parameters**, increasing total model size from **16,999,430** to **17,005,574** parameters.

Centroids are normalized during forward computation:

$$\hat{c}_i = \frac{c_i}{\|c_i\|}$$

For a query or key vector x , VayuSphere computes:

$$\hat{x} = \frac{x}{\|x\|}$$

and centroid similarity:

$$s_i = \hat{x}^\top \hat{c}_i$$

3.2 Top-k Centroid Selection

The strongest observed VayuSphere variant uses top-k centroid selection with $k = 8$. Instead of applying softmax over all centroids, VayuSphere selects the eight nearest centroids:

$$\mathcal{T}_k(x) = \text{TopK}(s_1, \dots, s_m)$$

and computes a local softmax only over this set:

$$w_i = \frac{\exp(s_i/\tau)}{\sum_{j \in \mathcal{T}_k(x)} \exp(s_j/\tau)}$$

This prevents every token from softly using every centroid and encourages sharper centroid specialization.

3.3 Scale Gate

The current best-performing VayuSphere implementation uses a scale gate. Given centroid similarity scores, the gate is centered and passed through a bounded nonlinearity:

$$g(x) = \tanh(s(x) - \mu_s)$$

The output vector is scaled as:

$$x' = x \cdot \exp(\alpha g(x))$$

where α controls gate strength. In the current proof run, $\alpha = 0.1$.

This means the scale factor is bounded roughly by:

$$\exp(-0.1) \leq \text{scale} \leq \exp(0.1)$$

or approximately:

$$0.90 \leq \text{scale} \leq 1.10$$

Thus, the current version is intentionally conservative.

3.4 RoPE Placement

Two VayuSphere placements were tested:

1. Pre-RoPE VayuSphere

$$Q, K \rightarrow \text{VayuSphere} \rightarrow \text{RoPE} \rightarrow \text{Attention}$$

1. Post-RoPE VayuSphere

$$Q, K \rightarrow \text{RoPE} \rightarrow \text{VayuSphere} \rightarrow \text{Attention}$$

Early reasoning suggested pre-RoPE placement might better preserve semantic clustering. However, experiments indicate that the stronger candidate is **post-RoPE + top-k8**, suggesting that VayuSphere may be more useful as a position-aware angular attention gate rather than a pure semantic router.

4. Experimental Setup

4.1 Model

The baseline model has:

Property	Value
Total parameters	16,999,430
VayuSphere parameters	+6,144
VayuSphere total parameters	17,005,574
Parameter overhead vs total	~0.036%

The model is embedding-heavy due to GPT-2 vocabulary size. The embedding table is a large fraction of total parameter count, so future experiments should also test larger models where the transformer body dominates more of the parameter budget.

4.2 Data

The experiments use a GPT-2-tokenized FineWeb-Edu subset:

Split	Tokens
Train	~10M
Validation	~500K

Training uses 10,000 steps. Evaluation during training used 10 eval points. Final re-evaluation used 200 validation batches:

$$200 \times 512 = 102,400 \text{ validation tokens}$$

This larger evaluation was introduced because the initial 20-batch eval was too small to reliably detect loss differences around 0.005 to 0.01 nats.

4.3 Compared Variants

The final proof test compared:

Variant	Description
<code>A_baseline</code>	Standard GPT baseline
<code>B_learned_temp</code>	Baseline with learned per-head attention temperature
<code>scale_all_postrope</code>	VayuSphere scale gate after RoPE, all centroids
<code>scale_topk8_postrope</code>	VayuSphere scale gate after RoPE, top-k=8, trained centroids
<code>scale_topk8_postrope_frozen_random</code>	Same as above, but centroids frozen randomly

The proof criterion required `scale_topk8_postrope` to beat all four controls on average across seeds.

5. Results

5.1 Single-Seed Post-RoPE Control Result

A strong single-seed result was observed for `scale_topk8_postrope` :

Variant	Loss	Delta vs baseline
<code>A_baseline</code>	6.254538	0.000000
<code>B_learned_temp</code>	6.254278	-0.000260
<code>scale_topk8_postrope</code>	6.247663	-0.006875
<code>scale_topk8_postrope_frozen_random</code>	6.252784	-0.001754

This suggests that trained centroids contribute more than frozen random centroids and learned temperature in this seed.

Key comparisons:

$$\Delta_{\text{trained vs baseline}} = -0.006875$$

$$\Delta_{\text{trained vs frozen}} = -0.005121$$

$$\Delta_{\text{trained vs learned temp}} = -0.006615$$

This is the strongest positive VayuSphere result so far.

5.2 Three-Seed Proof Re-evaluation

The final 3-seed proof test used 200-batch re-evaluation.

Variant	Mean loss	Mean delta vs baseline	Wins vs baseline
<code>A_baseline</code>	6.247002	0.000000	—
<code>scale_topk8_postrope</code>	6.248478	+0.001476	1 / 3
<code>scale_all_postrope</code>	6.249089	+0.002087	1 / 3
<code>scale_topk8_postrope_frozen_random</code>	6.250249	+0.003247	0 / 3
<code>B_learned_temp</code>	6.251026	+0.004023	1 / 3

The proof condition was not met because `scale_topk8_postrope` did not beat the baseline on average.

5.3 Per-Seed Results

Seed	Baseline	Top-k post-RoPE	Delta
123	6.239855	6.237685	-0.002170
42	6.250661	6.253013	+0.002352
777	6.250491	6.254736	+0.004245

The method wins on seed 123, but loses on seeds 42 and 777.

5.4 Control Comparisons

Although VayuSphere does not beat baseline on average, it does outperform the internal controls:

Comparison	Mean loss advantage of <code>scale_topk8_postrope</code>
vs learned temperature	~0.00255
vs frozen random centroids	~0.00177
vs all-centroid VayuSphere	~0.00061

This suggests the trained top-k centroid mechanism has signal, but the signal is weaker than seed-to-seed training noise at the current scale.

5.5 Throughput Cost

The main practical issue is runtime overhead.

Variant	Approximate throughput behavior
Baseline	Fastest
Learned temperature	Near baseline
VayuSphere variants	~1.7x to ~1.9x slower

The slowdown is not primarily caused by diagnostics. The core forward path itself introduces additional eager-mode tensor operations, including centroid similarity computation, top-k selection, softmax, centroid mixing, centering, and exponential scaling. These operations occur before scaled dot-product attention and may reduce the benefit of fused attention kernels.

6. Discussion

6.1 What Worked

The strongest evidence for VayuSphere is not yet baseline improvement. Instead, it is the control hierarchy:

trained top-k centroids > frozen random centroids

trained top-k centroids > learned temperature

top-k centroids > all centroids

This indicates that VayuSphere is not merely equivalent to learned attention temperature, and not merely random noise. The learned centroid mechanism contains useful signal.

6.2 What Did Not Work Yet

VayuSphere v0.2 does not yet robustly beat the GPT baseline across seeds. The likely reasons are:

1. **Weak scale-only gate:** The current gate only modulates Q/K magnitude by approximately $\pm 10\%$.
 2. **Small model regime:** The model is heavily embedding-dominated, so attention-body modifications may have limited influence.
 3. **No compute-saving path yet:** VayuSphere currently adds overhead rather than reducing FLOPs.
 4. **Seed noise:** Observed improvements are smaller than the variance across seeds.
 5. **Conservative alpha:** The current $\alpha = 0.1$ setting may underdrive the mechanism.
-

6.3 Post-RoPE Interpretation

The best single-seed result comes from post-RoPE VayuSphere. This suggests the mechanism may not be pure semantic clustering. Instead, it may act as a **position-aware angular gate**. Since RoPE encodes positional phase into query/key vectors, post-RoPE centroid similarity may capture a mixture of content and relative-position behavior.

This interpretation is important. For next-token prediction, positional structure is not noise; it is central to the task. A post-RoPE angular gate may therefore be more useful than a pre-RoPE semantic gate.

7. Limitations

This study has several limitations:

1. **Nano-scale model only:** Results may not transfer to larger models.
2. **Small training dataset:** 10M tokens is useful for rapid iteration but not sufficient for strong scaling claims.
3. **Limited seeds:** Three seeds are better than one, but still insufficient for strong statistical claims.

4. **No downstream evaluation:** Only language modeling loss/perplexity was measured.
 5. **No optimized kernel:** VayuSphere is implemented using ordinary PyTorch operations.
 6. **No demonstrated FLOP reduction:** The current method does not reduce attention compute; it adds modulation overhead.
 7. **Scale-only mechanism:** The tangent-space variant was tested but did not outperform the scale gate in the current setup.
-

8. Future Work

The current results suggest several next steps.

8.1 Alpha Sweep

The current scale gate uses $\alpha = 0.1$. Future experiments should test:

$$\alpha \in \{0.05, 0.10, 0.20, 0.40\}$$

on the post-RoPE top-k8 pipeline.

8.2 Target Sweep

The current best setting applies VayuSphere to both Q and K. Since attention logits depend on:

$$(q \cdot k)$$

scaling both Q and K compounds the gate multiplicatively. Future experiments should test:

- Q-only,
- K-only,
- QK.

8.3 Larger Model Test

A larger model such as an 80M or small Colab/T4 configuration should be tested. This would reduce the dominance of the embedding table and give attention-body modifications more room to matter.

8.4 Improved Angular Adapter

The current scale-only mechanism should be extended with stronger angular updates:

$$x' = x + \alpha \|x\| \cdot t$$

where t is a tangent-space correction toward the centroid mixture. Earlier tangent experiments were not yet successful, but the idea remains promising.

8.5 Kernel Optimization

If the mechanism becomes robust, optimize runtime using:

- `torch.compile`,
- profiling with `torch.profiler`,
- replacing `topk` /scatter-heavy paths,
- fused custom kernels,
- Triton implementation.

Optimization should come after the mechanism reliably beats baseline.

9. Conclusion

VayuSphere introduces a lightweight hyperspherical centroid adapter for query-key modulation in decoder transformers. The method adds negligible parameters and shows evidence of learned centroid signal: trained top-k post-RoPE centroids outperform frozen random centroids, learned temperature, and all-centroid variants on average.

However, the current VayuSphere v0.2 implementation does **not** yet robustly outperform the GPT baseline across seeds. The best variant, `scale_topk8_postrope`, achieves mean loss **6.2485**, while the baseline achieves mean loss **6.2470** under 200-batch re-evaluation. Therefore, the current result is best understood as a partial positive: VayuSphere has measurable internal signal but requires stronger mechanism design or tuning before it can be claimed as a baseline-beating architecture.

The next decisive experiment is a post-RoPE top-k8 alpha and target sweep, followed by a larger-model validation if a stronger setting is found.

References

Vaswani et al. "Attention Is All You Need." 2017.

Su et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding." 2021.

Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." 2022.

Touvron et al. "LLaMA: Open and Efficient Foundation Language Models." 2023.

Radford et al. "Language Models are Unsupervised Multitask Learners." 2019.

Zhang and Sennrich. "Root Mean Square Layer Normalization." 2019.

Shazeer. "GLU Variants Improve Transformer." 2020.